

Dominando los Comandos SQL

Sintaxis esencial para crear, modificar y estructurar bases de datos relacionales.

¿Qué es DDL?

Data Definition Language (Lenguaje de Definición de Datos) es el conjunto de comandos SQL que te permite crear, modificar y eliminar la estructura de tablas y bases de datos.

La analogía perfecta

DDL es como el **plano de una casa**: define dónde van las paredes, puertas y ventanas. Antes de guardar datos, necesitas definir *dónde* y *cómo* se almacenarán.



CREATE



ALTER



DROP

Los Tipos de Datos Más Comunes

Los Ladrillos del SQL

Antes de crear cualquier tabla, debes conocer qué tipo de dato almacenará cada columna. Elegir el tipo correcto garantiza eficiencia y precisión en tu base de datos.

Tipo de Dato SQL	¿Qué almacena?	Ejemplo Real
INT (INTEGER)	Números enteros sin decimales	Edad: 25, ID: 1001
VARCHAR(N)	Texto variable hasta N caracteres	Nombre: "Juan García"
DECIMAL(P,S)	Números con decimales precisos	Precio: 99.99, Sueldo: 2500.50
DATE	Fechas en formato YYYY-MM-DD	Nacimiento: 1995-03-15
BOOLEAN	Valores verdadero/falso (1/0)	Activo: TRUE, Eliminado: FALSE

Tu Primera Tabla — CREATE TABLE

Sintaxis básica para crear una tabla simple

```
CREATE TABLE Clientes (  
  -- Columna: nombre_columna TIPO_DATO restricciones  
  id_cliente    INT      NOT NULL,  
  nombre        VARCHAR(100) NOT NULL,  
  email         VARCHAR(100),  
  fecha_registro DATE     DEFAULT CURRENT_DATE,  
  activo        BOOLEAN   DEFAULT TRUE  
);  
-- NOT NULL: Campo obligatorio (no puede quedar vacío)  
-- DEFAULT: Valor automático si no se especifica  
-- ;      Cierra la sentencia SQL
```

NOT NULL

Obliga a que el campo siempre tenga un valor.
Ideal para datos críticos como nombre o ID.

DEFAULT

Asigna un valor automático cuando no se proporciona uno. Ahorra trabajo en inserciones.

El punto y coma ;

Toda sentencia SQL debe terminar con ; para indicar el fin del comando.

Asegurando la Identidad — Llave Primaria (PK)

La **Primary Key** identifica de forma única cada fila de una tabla. No puede repetirse ni ser nula.

Forma 1: Inline (en la misma línea)

```
CREATE TABLE Clientes (  
  -- PK declarada junto al tipo de dato  
  id_cliente INT PRIMARY KEY NOT NULL,  
  nombre VARCHAR(100) NOT NULL  
);
```

 Útil para tablas pequeñas o prototipos rápidos.

Forma 2: Table Constraint — RECOMENDADO

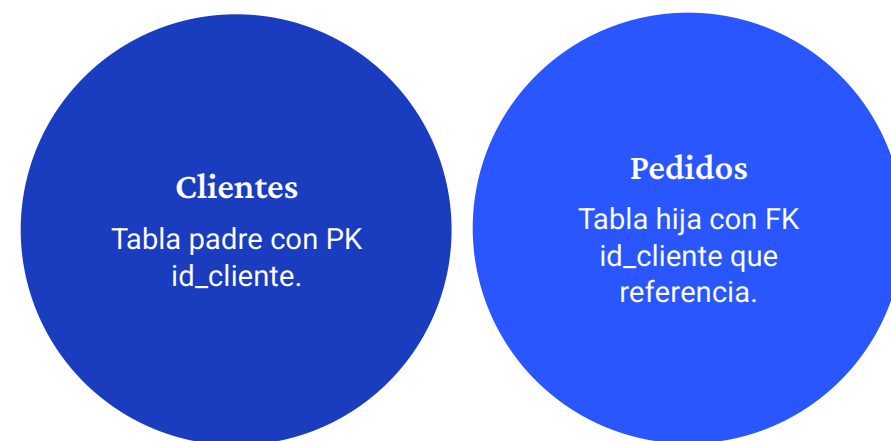
```
CREATE TABLE Clientes (  
  id_cliente INT NOT NULL,  
  nombre VARCHAR(100) NOT NULL,  
  -- PK declarada al final, separada de las columnas  
  PRIMARY KEY (id_cliente)  
);  
-- Mejor para proyectos grandes:  
-- más legible y flexible
```

 Permite definir PKs compuestas (más de una columna) con facilidad.

Conectando Tablas — Llave Foránea (FK)

Sintaxis para relacionar dos tablas

```
CREATE TABLE Pedidos (  
  id_pedido INT PRIMARY KEY NOT NULL,  
  id_cliente INT NOT NULL,  
  fecha_pedido DATE NOT NULL,  
  
  -- FK: apunta a la PK de la tabla Clientes  
  FOREIGN KEY (id_cliente)  
    REFERENCES Clientes(id_cliente)  
);  
  
-- id_cliente en Pedidos = FK (tabla hija)  
-- id_cliente en Clientes = PK (tabla padre)
```



❏ La FK garantiza **integridad referencial**: no puedes crear un pedido para un cliente que no existe.

Modificando la Estructura — ALTER TABLE (Parte 1)

Añadir nuevos campos a una tabla existente

Cuando tu base de datos ya tiene datos y necesitas agregar columnas, `ALTER TABLE` te permite hacerlo sin perder información.

-- Agregar una sola columna nueva

```
ALTER TABLE Clientes
```

```
ADD COLUMN telefono VARCHAR(15);
```

-- Agregar múltiples columnas en una sola sentencia

```
ALTER TABLE Clientes
```

```
ADD COLUMN ciudad VARCHAR(50),
```

```
ADD COLUMN pais VARCHAR(50);
```

-- Agregar columna con restricciones incluidas

```
ALTER TABLE Clientes
```

```
ADD COLUMN fecha_nacimiento DATE NOT NULL DEFAULT '1990-01-01';
```

-- DEFAULT obligatorio cuando la tabla ya tiene filas y la columna es NOT NULL

Una columna

```
ADD COLUMN nombre TIPO;
```

Múltiples columnas

Separa cada columna con `,` antes del `;` final.

Con restricciones

Incluye `NOT NULL`, `DEFAULT` o `CHECK` al agregar.

Corrigiendo Errores — ALTER TABLE (Parte 2)

Cambiar el tipo de dato de una columna

-- En MySQL:

```
ALTER TABLE Clientes  
MODIFY COLUMN telefono VARCHAR(20);
```

-- En PostgreSQL / SQL Server:

```
ALTER TABLE Clientes  
ALTER COLUMN telefono VARCHAR(20);
```

-- Útil cuando el campo original era muy corto
-- y necesitas más espacio para los datos

Eliminar una columna que ya no se necesita

-- Eliminar columna "ciudad" de la tabla

```
ALTER TABLE Clientes  
DROP COLUMN ciudad;
```

--

Agregando Reglas de Control — CHECK

Validar datos antes de que entren a la base de datos

```
CREATE TABLE Empleados (  
  id_empleado INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  
  -- CHECK inline: edad debe ser 18 o más  
  edad INT CHECK (edad >= 18),  
  
  -- CHECK inline: sueldo debe ser positivo  
  sueldo DECIMAL(10,2) CHECK (sueldo > 0),  
  
  email VARCHAR(100),  
  
  -- CONSTRAINT nombrado: más fácil de identificar  
  CONSTRAINT chk_sueldo_minimo  
    CHECK (sueldo >= 1000)  
);  
  
-- CHECK: valida condiciones al insertar/actualizar  
-- CONSTRAINT: nombra la regla para identificarla después
```

1 CHECK inline

Se escribe junto a la columna. Rápido para reglas simples.

2 CONSTRAINT nombrado

Se declara al final con un nombre. Fácil de identificar y eliminar después.

3 Cuándo usar CHECK

Rangos de edad, precios positivos, porcentajes entre 0 y 100, estados válidos.

Borrón y Cuenta Nueva — DROP vs TRUNCATE

Diferencia crítica entre dos comandos de eliminación

Comando	¿Qué hace?	Riesgo	Sintaxis
DROP TABLE	Elimina la tabla completa: estructura + datos + índices	IRREVERSIBLE ⚠	DROP TABLE Clientes;
TRUNCATE TABLE	Vacía todos los datos pero mantiene la estructura de la tabla	Más seguro ✓	TRUNCATE TABLE Clientes;

-- Eliminar la tabla COMPLETAMENTE (estructura + datos)
DROP TABLE Clientes;

-- Solo vaciar los datos (la tabla sigue existiendo)
TRUNCATE TABLE Clientes;

⊗ **Regla de oro:** Usa TRUNCATE para limpiar datos temporales. Usa DROP solo si necesitas eliminar la tabla completamente y de forma permanente.

Hoja de Ruta SQL — Cheat Sheet

Todo lo que aprendiste en una sola vista

1

CREATE TABLE

Crear tabla con columnas y tipos de datos (INT, VARCHAR, DATE...)

2

PRIMARY KEY

Identificador único e irrepetible de cada fila. Nunca nulo.

3

FOREIGN KEY

Conecta tablas relacionadas garantizando integridad referencial.

4

ALTER TABLE

ADD columnas · MODIFY tipo de dato · DROP columna innecesaria.

5

CHECK & CONSTRAINT

Validar datos con condiciones antes de insertar o actualizar.

6

TRUNCATE vs DROP

Vaciar datos vs. eliminar tabla completa. ¡Conoce la diferencia!



Próximo paso: Domina INSERT, UPDATE, SELECT y DELETE para manipular los datos dentro de tus tablas.